

Detección de cryptojacking en Windows 11 con Wazuh

Laboratorio de detección por comportamiento: telemetría de rendimiento, reglas propias y correlación mapeada a MITRE ATT&CK T1496

Resumen del proyecto

Laboratorio personal con Wazuh 4.14 sobre Parrot OS que monitoriza un Windows 11 en un segmento de red aislado. Cada 60 segundos, un script de PowerShell basado en clases CIM (independiente del idioma del sistema) envía al manager, en formato JSON, el uso de CPU, memoria, disco y red, junto con el proceso que más CPU consume.

Las reglas propias avisan cuando se supera un umbral, y la regla 100110 (MITRE T1496) correlaciona cinco muestras de CPU alta del mismo proceso en diez minutos para distinguir un minero de un pico puntual de trabajo.

Autor: Diego Sanz Rodríguez

Versión: 1.1 · 26 de septiembre de 2026

Contacto: [\[GitHub / LinkedIn\]](#)

Índice

1. Resumen.....	3
Ficha del proyecto.....	3
2. Contexto y objetivo.....	4
3. Arquitectura.....	5
3.1 Topología.....	5
3.2 Puertos.....	5
3.3 Flujo de los datos.....	6
4. Decisiones técnicas.....	7
4.1 Clases CIM en lugar de Get-Counter.....	7
4.2 Log en ASCII en lugar de UTF-8.....	7
4.3 Umbrales expresados como expresiones regulares.....	7
4.4 Correlación con same_field: el núcleo de la detección.....	8
4.5 Escritura del log compartida con el agente.....	8
4.6 Otras decisiones.....	8
5. Implementación.....	9
5.1 Recogida de métricas en el endpoint.....	9
5.2 Envío al manager.....	9
5.3 Reglas de detección.....	10
5.4 Repositorio.....	10
6. Validación y resultados.....	11
6.1 Validación de las reglas con wazuh-logtest.....	11
6.2 Prueba de carga controlada.....	11
6.3 Observaciones de la prueba.....	13
6.4 Qué demuestra y qué no.....	13
7. Problemas encontrados y cómo los resolví.....	14
8. Limitaciones y siguientes pasos.....	17
9. Tecnologías y aprendizajes.....	18
Lo que me llevo.....	18
Anexo: cómo reproducir el laboratorio.....	19

Historial de versiones

Versión	Fecha	Cambios
1.0	23/09/2026	Primera versión: arquitectura, decisiones técnicas, reglas 100100–100110 y validación inicial.
1.1	26/09/2026	Documento reorganizado. La regla 100110 pasa a correlacionar el grupo <code>cpu_high</code> . Se añaden los problemas más recientes (escritura bloqueada del log, PowerShell de 32 bits, CPU de la VM por debajo del 95 %, desfase horario y codificación de la consola), las observaciones de la prueba de carga y las limitaciones actualizadas.

1. Resumen

Este proyecto es un laboratorio de detección de cryptojacking en Windows 11 construido con Wazuh. Cubre el proceso completo: el montaje del SIEM, la recogida de telemetría en el endpoint, la escritura de reglas propias y su validación con pruebas controladas. No he usado reglas de terceros; la telemetría, los umbrales y la correlación están hechos desde cero para este caso de uso.

La idea principal es detectar por **comportamiento** y no por firmas. En lugar de avisar cada vez que la CPU sube, la regla 100110 exige que un mismo proceso mantenga la CPU alta durante unos cinco minutos, que es lo que diferencia a un minero de una compilación, un juego o una actualización. En las pruebas, la alerta de nivel 12 aparece unos cinco minutos después de empezar la carga y queda asociada a la técnica T1496 (Resource Hijacking).

Ficha del proyecto

Objetivo	Detectar un uso sostenido y anómalo de recursos (cryptojacking) en equipos Windows 11
Entorno	Laboratorio aislado (192.168.1.0/24): Parrot OS como manager y máquina atacante, Windows 11 como víctima
Versiones	Wazuh 4.14 (manager) · agente 4.14.7 · PowerShell 5.1
Detección	7 reglas propias (100100–100110) y correlación por proceso mapeada a MITRE ATT&CK T1496
Estado	Funcional y validado en laboratorio; umbrales pendientes de calibrar con uso normal
Repositorio	[enlace al repositorio con los scripts y las reglas]
Autor	Diego Sanz Rodríguez · septiembre de 2026

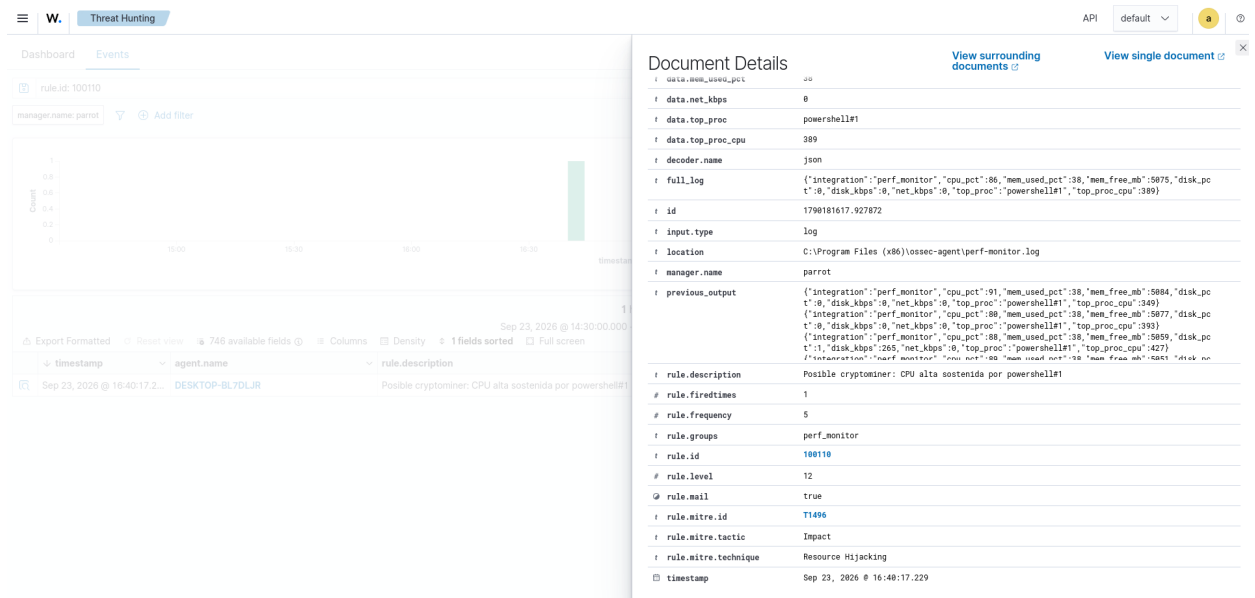


Figura 1. Alerta 100110 en Threat Hunting: nivel 12, técnica MITRE T1496 (Resource Hijacking) y, en previous_output, las muestras del mismo proceso que la dispararon.

2. Contexto y objetivo

Con el auge de las criptomonedas han aparecido ataques que aprovechan equipos ajenos para minar. El cryptojacking normalmente no roba datos ni cifra ficheros, pero consume CPU, electricidad y vida útil del hardware en beneficio de un tercero. En una empresa esto es un coste directo y, además, es una señal de que alguien ha conseguido ejecutar código en el equipo, así que no conviene tratarlo como un problema menor.

MITRE ATT&CK recoge esta técnica como **T1496 (Resource Hijacking)**. Su rasgo más característico es que el minero necesita mucho procesamiento durante mucho tiempo. Por eso un filtro simple del tipo «CPU > 90 %» no sirve: saltaría con cualquier tarea pesada. Hay que observar el equipo a lo largo del tiempo y fijarse en qué proceso es el responsable del consumo.

Con esa idea, el objetivo del proyecto es construir con Wazuh 4.14 un conjunto de reglas que distinga un pico normal de trabajo de un consumo sostenido compatible con un minero, y que funcione en cualquier Windows 11 sin depender del idioma del sistema ni de software adicional.

Referencia: <https://attack.mitre.org/techniques/T1496/>

Alcance y consideraciones éticas

Todo el trabajo se ha realizado en un laboratorio propio y aislado. Las pruebas de detección se hacen con carga sintética generada por un script que satura la CPU de forma controlada, no con muestras reales de malware. El objetivo es exclusivamente defensivo: construir y validar la detección.

3. Arquitectura

3.1 Topología

El laboratorio está formado por dos máquinas virtuales en un segmento interno aislado (192.168.1.0/24). Parrot OS hace de manager de Wazuh y también de máquina atacante; Windows 11 es el equipo monitorizado. Solo Parrot tiene una segunda interfaz con salida a internet, que uso para la gestión.

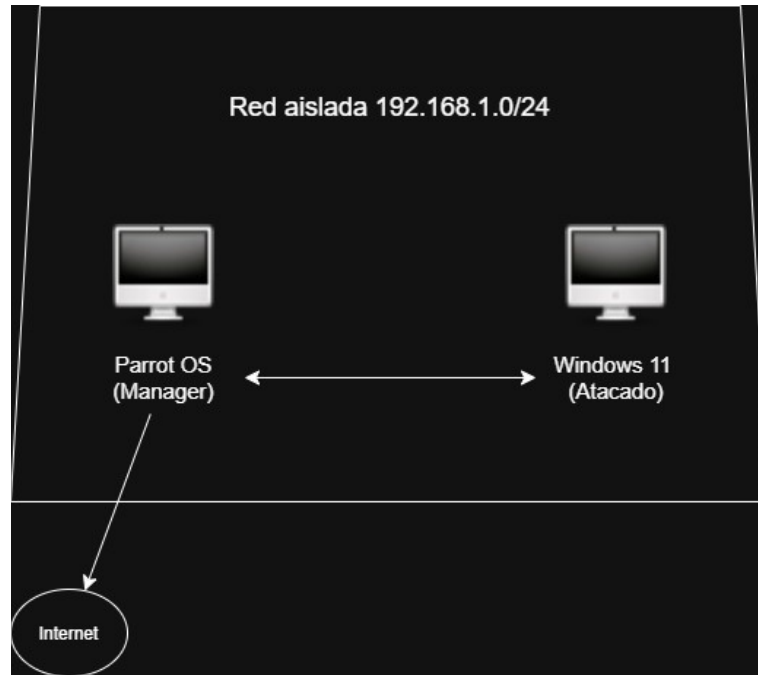


Figura 2. Topología del laboratorio: manager y endpoint en un segmento interno aislado.

Máquina	Rol	Software
Parrot OS	Manager de Wazuh y máquina atacante	Wazuh 4.14 todo-en-uno (indexer + server + dashboard)
Windows 11	Endpoint monitorizado (víctima)	Agente Wazuh 4.14.7 + script de telemetría en PowerShell

3.2 Puertos

Para que Wazuh funcione tienen que estar accesibles los siguientes puertos en el manager:

Puerto	Uso	Exposición
1514/tcp	Comunicación agente → manager (envío de eventos)	Solo segmento interno
1515/tcp	Enrolamiento (registro) de agentes	Solo segmento interno
443/tcp	Dashboard web de Wazuh	Solo interfaz de gestión

El segmento interno no tiene salida a internet, así que el tráfico entre el agente y el manager no sale del laboratorio.

3.3 Flujo de los datos

El script de PowerShell se ejecuta cada 60 segundos en el endpoint y escribe una línea JSON en un log local. El agente lee ese log y lo envía al manager, donde el decoder JSON convierte cada clave en un campo consultable. Las reglas evalúan esos campos y, cuando alguna coincide, la alerta aparece en `alerts.log` y en el dashboard.

```
perf-monitor.ps1 -> perf-monitor.log -> agente (localfile)
                    -> manager: decoder JSON -> reglas 100100-100110
                    -> alerts.log + dashboard
```

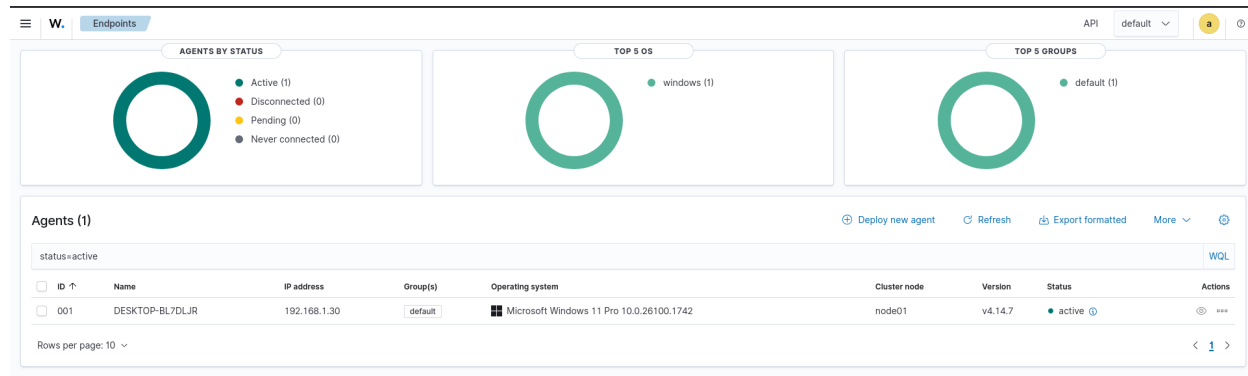


Figura 3. Apartado Endpoints del dashboard: el agente Windows 11 (v4.14.7, IP 192.168.1.30) aparece como activo.

4. Decisiones técnicas

En este apartado explico las decisiones que más han influido en el diseño, qué problema resolvía cada una y qué coste tiene. Documentar el coste me parece tan importante como la decisión, porque es lo que marca los límites de la detección.

4.1 Clases CIM en lugar de Get-Counter

Problema. La documentación oficial de Wazuh lee los contadores de rendimiento con `Get-Counter`, pero las rutas de esos contadores están traducidas al idioma del sistema. En un Windows en español, `\Processor(_Total)\% Processor Time` pasa a llamarse `\Procesador(_Total)\% de tiempo de procesador`, y el script copiado tal cual falla sin avisar.

Decisión. Leer los mismos contadores a través de las clases CIM `Win32_PerfFormattedData_*`, cuyas propiedades se llaman igual en cualquier idioma. El dato es el mismo, pero el script funciona en cualquier Windows 11, que es justo lo que necesita un SIEM que quiere ser «universal».

Coste. La consulta de procesos devuelve todos los procesos del sistema en cada muestra, lo que pesa algo más que pedir un único contador. Con un intervalo de 60 segundos no se nota.

4.2 Log en ASCII en lugar de UTF-8

Problema. En PowerShell 5.1, la codificación UTF-8 escribe un BOM (unos bytes invisibles al inicio del fichero). El agente los lee como parte del primer JSON y no consigue parsearlo.

Decisión. Escribir en ASCII. PowerShell 7 tiene una opción de UTF-8 sin BOM, pero el agente invoca PowerShell 5.1, que es la versión que trae Windows 11 de serie, y no quise añadir una dependencia solo por esto.

Coste. ASCII no admite tildes ni caracteres no latinos, así que un proceso con esos caracteres en el nombre aparecería con «?». Los ejecutables casi nunca los usan, de modo que lo acepté como limitación documentada.

4.3 Umbrales expresados como expresiones regulares

Problema. Wazuh no compara números al evaluar campos, compara texto. Una condición como «CPU mayor que 80» no se puede escribir tal cual, y es la causa más habitual de reglas que nunca se disparan.

Decisión. Traducir cada umbral a una expresión regular anclada que coincida exactamente con el rango de valores:

Regla	Condición	Expresión regular
100101	CPU 80-94 %	<code>^(8[0-9] 9[0-4])\$</code>
100102	CPU 95-100 %	<code>^(9[5-9] 100)\$</code>
100103	Memoria libre < 1000 MB	<code>^[0-9]{1,3}\$</code>
100104	Disco ≥ 90 %	<code>^(9[0-9] 100 [1-9][0-9]{2,})\$</code>
100105	Red ≥ 10 000 KB/s	<code>^[0-9]{5,}\$</code>

Los rangos de CPU no se solapan, así que cada muestra cae en un único nivel de severidad. Para que esto funcione, el script convierte todas las métricas a enteros: un valor como 85.3 rompería las expresiones.

Coste. Las expresiones regulares son frágiles. Sin anclas, `[0-9]{1,3}` coincidiría con cualquier número (4200 contiene tres dígitos seguidos) y la alerta de memoria baja saltaría en cada muestra. Por eso cada regla tiene su muestra de prueba en `wazuh-logtest`. Además, cambiar un umbral obliga a reescribir la expresión, lo que hace más lenta la calibración.

4.4 Correlación con `same_field`: el núcleo de la detección

Problema. Una regla de umbral simple genera mucho ruido, porque compilar, jugar o una actualización de Windows ponen la CPU al 100 %. Lo que caracteriza a un minero no es el pico, sino la persistencia: el mismo proceso ocupando la CPU durante mucho tiempo.

Decisión. La regla 100110 solo se dispara con 5 muestras de CPU alta en 10 minutos atribuidas al mismo proceso. Con muestreo cada 60 segundos, eso equivale a unos 5 minutos de carga sostenida. Cuenta cualquier muestra del grupo `cpu_high`, es decir, tanto las de la 100101 (80–94 %) como las de la 100102 (95–100 %). `same_field` es la pieza clave: sin él, una actualización seguida de una compilación y un juego sumarían cinco muestras y parecerían un minero. La regla tiene nivel 12 y está mapeada a MITRE ATT&CK T1496.

Cambio en la versión 1.1

Al principio la 100110 solo correlacionaba la 100102 (95–100 %). En las pruebas, la CPU de la máquina virtual nunca pasó del 91 %, así que la regla no llegaba a dispararse aunque la carga era sostenida (ver apartado 7.1). La cambié para que correlacione el grupo `cpu_high` completo.

Coste. Tiene varios:

- La detección tarda unos 5 minutos. Me parece aceptable porque el cryptojacking es un robo de recursos continuado, no una acción puntual.
- Al bajar el umbral efectivo al 80 %, aumenta el riesgo de falsos positivos con tareas legítimas largas. Por eso la calibración con uso normal es el siguiente paso.
- Un minero que limite su consumo por debajo del 80 % no la dispara.
- El proceso se identifica por su nombre de instancia, no por su ruta. Un minero renombrado como `svchost` aparecería como `svchost`. Además, cuando hay varios procesos con el mismo nombre, Windows los numera (`powershell#1`, `#2...`) y esa numeración puede cambiar, que es la razón de que la prueba de carga use un único proceso. Añadir el PID y la ruta del ejecutable a la telemetría resolvería ambas cosas.

4.5 Escritura del log compartida con el agente

Problema. El agente mantiene el fichero `perf-monitor.log` abierto de forma continua para leerlo. `Add-Content` pide un modo de acceso que no es compatible con ese fichero ya abierto y la escritura falla.

Decisión. Abrir el fichero con `[System.IO.File]::Open(...)` indicando `FileShare.ReadWrite`, de forma que el script y el agente puedan usar el fichero a la vez.

Coste. El script queda algo más largo y menos legible que con un simple `Add-Content`, pero es la única forma fiable que encontré. El detalle de cómo lo descubrí está en el apartado 7.2.

4.6 Otras decisiones

- **Separar ejecución y envío.** El wodle solo lanza el script y el `localfile` es quien lee y envía el log. `ignore_output` en `yes` evita que cada muestra llegue dos veces.
- **Regla base de nivel 0.** Clasifica toda la telemetría sin generar alertas y de ella cuelgan las demás reglas, así que el flujo normal de una muestra por minuto no llena el dashboard de ruido.
- **Ruta completa a PowerShell de 64 bits.** El agente es un proceso de 32 bits; indicando la ruta de `System32` evito que Windows lo redirija al PowerShell de 32 bits (apartado 7.3).

5. Implementación

5.1 Recogida de métricas en el endpoint

El script `perf-monitor.ps1` se instala junto al agente y se ejecuta cada 60 segundos. Consulta las clases CIM de rendimiento, monta un objeto con las métricas y lo escribe como una única línea JSON:

Campo	Qué contiene
<code>integration</code>	Etiqueta fija «perf_monitor»: permite que la regla base identifique la telemetría
<code>cpu_pct</code>	Porcentaje de CPU total (instancia _Total)
<code>mem_used_pct / mem_free_mb</code>	Memoria usada en porcentaje y memoria libre en MB
<code>disk_pct / disk_kbps</code>	Porcentaje de tiempo de disco activo y KB/s transferidos
<code>net_kbps</code>	KB/s totales, excluyendo loopback y adaptadores virtuales
<code>top_proc / top_proc_cpu</code>	Proceso que más CPU consume y su consumo: es la clave de la correlación

Una muestra tiene este aspecto:

```
{"integration":"perf_monitor","cpu_pct":97,"mem_used_pct":72,"mem_free_mb":850,"disk_pct":12,"disk_kbps":400,"net_kbps":120,"top_proc":"xmrig","top_proc_cpu":95}
```

La parte final del script, que escribe la muestra en el log, quedó así después de resolver el problema de acceso al fichero (apartado 7.2):

```
$fs = [System.IO.File]::Open($logFile, [System.IO.FileMode]::Append,  
    [System.IO.FileAccess]::Write, [System.IO.FileShare]::ReadWrite)  
$sw = New-Object System.IO.StreamWriter($fs, [System.Text.Encoding]::ASCII)  
$sw.WriteLine($json)  
$sw.Flush(); $sw.Close(); $fs.Close()
```

5.2 Envío al manager

En el `ossec.conf` del agente, el módulo wodle se encarga de ejecutar el script y el localfile de leer y enviar el log. Están separados a propósito: si el wodle enviase también su salida, cada muestra llegaría duplicada, de ahí `ignore_output`.

```
<wodle name="command">  
  <disabled>no</disabled>  
  <tag>perf-monitor</tag>  
  <command>C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe  
    -ExecutionPolicy Bypass  
    -File "C:\Program Files (x86)\ossec-agent\perf-monitor.ps1"</command>  
  <interval>60s</interval>  
  <ignore_output>yes</ignore_output>  
  <run_on_start>yes</run_on_start>  
  <timeout>30</timeout>  
</wodle>  
  
<localfile>  
  <location>C:\Program Files (x86)\ossec-agent\perf-monitor.log</location>  
  <log_format>json</log_format>  
</localfile>
```

5.3 Reglas de detección

Todas las reglas cuelgan de una regla base de nivel 0 que clasifica la telemetría sin generar alertas. Sobre ella se apoyan las reglas de umbral y, encima de estas, la de correlación. Las dos reglas de CPU comparten el grupo `cpu_high`, que es el que usa la 100110:

```
<rule id="100102" level="8">
  <if_sid>100100</if_sid>
  <field name="cpu_pct" type="pcre2">^(9[5-9]|100)$</field>
  <description>CPU crítica: $(cpu_pct)% - proceso dominante: $(top_proc)</description>
  <group>cpu_high,</group>
</rule>

<rule id="100110" level="12" frequency="5" timeframe="600">
  <if_matched_group>cpu_high</if_matched_group>
  <same_field>top_proc</same_field>
  <description>Posible cryptominer: CPU alta sostenida por $(top_proc)</description>
  <mitre><id>T1496</id></mitre>
</rule>
```

El conjunto completo queda así:

ID	Nivel	Condición
100100	0	Cualquier muestra de perf_monitor (solo clasifica, no alerta)
100101	5	CPU entre 80 y 94 % (grupo cpu_high)
100102	8	CPU entre 95 y 100 % (grupo cpu_high)
100103	5	Memoria libre por debajo de 1000 MB
100104	5	Actividad de disco igual o superior al 90 %
100105	5	Tráfico de red igual o superior a 10 000 KB/s
100110	12	5 coincidencias del grupo cpu_high del mismo proceso en 600 s (MITRE T1496)

5.4 Repositorio

Todo lo necesario para reproducir el laboratorio está en el repositorio, con un README que indica qué archivo va en cada máquina:

```
siem-win11-lab/
├── README.md                               índice y comandos por fase
├── manager/instalar-wazuh-manager.sh       instalación del stack en Parrot
├── manager/local_rules.xml                 reglas 100100-100110
├── agente/instalar-agente.ps1             instalación del agente Windows
├── agente/perf-monitor.ps1                 telemetría CIM cada 60 s
├── agente/ossec-conf-perf-monitor.xml      wodle + localfile
├── pruebas/logtest-muestras.txt            una muestra JSON por regla
├── pruebas/stress-cpu-100102-100110.ps1    CPU sostenida (correlación)
├── pruebas/stress-cpu-100101.ps1           CPU 80-94 % corta
├── pruebas/stress-memoria-100103.ps1       memoria libre < 1000 MB
├── pruebas/stress-disco-100104.ps1         disco > 90 %
├── pruebas/stress-red-100105.ps1          red > 10 MB/s
└── docs/                                  esta documentación y capturas
```

6. Validación y resultados

6.1 Validación de las reglas con wazuh-logtest

Antes de generar carga real conviene comprobar que cada regla coincide con lo que debe. wazuh-logtest permite inyectar una línea JSON y ver en qué regla cae. Así se separa un problema de las reglas de un problema de envío: si la muestra cae en la regla base o en ninguna, el fallo está en el campo, no en el agente.

Muestra inyectada	Regla esperada
CPU 12 %, memoria 4200 MB libres, disco 3 %, red 40 KB/s	100100 (nivel 0, sin alerta)
CPU 85 %	100101
CPU 97 %, top_proc xmrig	100102
Memoria libre 850 MB	100103
Disco 95 %	100104
Red 15 000 KB/s	100105
Cinco muestras de CPU ≥ 80 % con el mismo top_proc en la misma sesión	100110

```

**Phase 3: Completed filtering (rules).
  id: '100102'
  level: '8'
  description: 'CPU crítica: 97% – proceso dominante: xmrig'
  groups: ['perf_monitor', 'cpu_high']
  firetimes: '1'
  mail: 'False'
**Alert to be generated.

```

Figura 4. wazuh-logtest: la muestra con CPU al 97 % cae en la regla 100102 (grupos perf_monitor y cpu_high) y genera alerta.

6.2 Prueba de carga controlada

La segunda prueba valida la cadena completa, de extremo a extremo. El script de carga crea un hilo por núcleo lógico dentro de un único proceso de PowerShell. Así imita el perfil de un minero real y, sobre todo, mantiene estable el valor de top_proc, que es lo que necesita same_field para acumular coincidencias. La lancé durante 8 minutos:

```
.\stress-cpu-100102-100110.ps1 -DurationMinutes 8
```

```

PS C:\Users\atacado> cd .\Desktop\
PS C:\Users\atacado\Desktop> .\stress-cpu-100102-100110.ps1 -DurationMinutes 8
Núcleos: 6 | Hilos de carga: 12 | PID: 3984
PS C:\Users\atacado\Desktop>

```

Figura 5. Lanzamiento de la prueba: 6 núcleos lógicos, 12 hilos de carga y un único proceso (PID 3984).

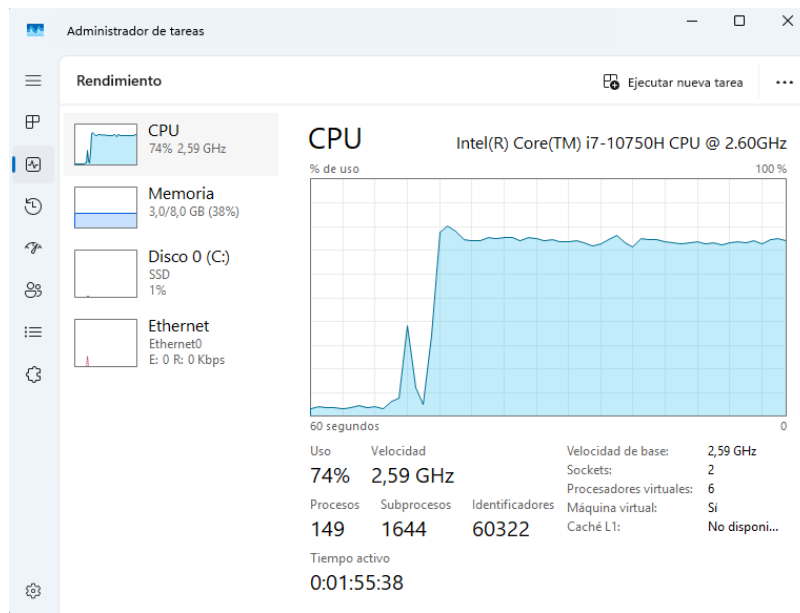


Figura 6. Administrador de tareas de la víctima durante la prueba: la CPU sube de golpe y se mantiene alta, pero sin llegar al 100 %.

En la telemetría se ve la carga en tiempo real. La CPU se mantuvo entre el 80 y el 91 % y el proceso dominante fue siempre powershell#1 (Windows le añadió «#1» porque había otra instancia de PowerShell abierta; lo importante es que el nombre no cambió durante la prueba):

```
PS C:\Users\atacado> Get-Content "C:\Program Files (x86)\ossec-agent\perf-monitor.log" -Tail 1 -Wait
{"integration":"perf_monitor","cpu_pct":72,"mem_used_pct":38,"mem_free_mb":5058,"disk_pct":0,"disk_kbps":30,"net_kbps":0,
,"top_proc":"powershell#1","top_proc_cpu":317}
{"integration":"perf_monitor","cpu_pct":80,"mem_used_pct":38,"mem_free_mb":5077,"disk_pct":0,"disk_kbps":0,"net_kbps":0,
,"top_proc":"powershell#1","top_proc_cpu":393}
{"integration":"perf_monitor","cpu_pct":91,"mem_used_pct":38,"mem_free_mb":5084,"disk_pct":0,"disk_kbps":0,"net_kbps":0,
,"top_proc":"powershell#1","top_proc_cpu":349}
{"integration":"perf_monitor","cpu_pct":86,"mem_used_pct":38,"mem_free_mb":5075,"disk_pct":0,"disk_kbps":0,"net_kbps":0,
,"top_proc":"powershell#1","top_proc_cpu":389}
{"integration":"perf_monitor","cpu_pct":91,"mem_used_pct":38,"mem_free_mb":5077,"disk_pct":0,"disk_kbps":0,"net_kbps":0,
,"top_proc":"powershell#1","top_proc_cpu":404}
{"integration":"perf_monitor","cpu_pct":80,"mem_used_pct":38,"mem_free_mb":5079,"disk_pct":0,"disk_kbps":0,"net_kbps":1,
,"top_proc":"powershell#1","top_proc_cpu":416}
```

Figura 7. Salida de perf-monitor.log en directo: una muestra por minuto con el mismo top_proc.

En el manager, cada muestra por encima del 80 % generó una alerta 100101 de nivel 5:

```
[*]-[root@parrot]-[~]
#sudo tail -f /var/ossec/logs/alerts/alerts.log
win.system.threadId: 2044
win.system.channel: System
win.system.computer: DESKTOP-BL7DLJR
win.system.severityValue: INFORMATION
win.system.message: "El tipo de inicio del servicio Servicio de transferencia inteligente en segundo plano (BITS) se cambió de inicio automático a inicio por solicitud."
win.eventdata.param1: Servicio de transferencia inteligente en segundo plano (BITS)
win.eventdata.param2: inicio automático
win.eventdata.param3: inicio por solicitud
win.eventdata.param4: BITS

** Alert 1790182704.947991: ~ syslog,sudo,pci_dss_10.2.5,pci_dss_10.2.2,pgp13.7.6,pgp13.7.8,pgp13.7.13,gdpr_IV_32.2,hipaa_164.312.b,nist_800_53_AU.14,nist_800_53_AC.7,nist_800_53_AC.6,
tsc_CC6.8,tsc_CC7.2,tsc_CC7.3,
2026 Sep 23 18:58:24 parrot->journald
Rule: 5402 (level 3) -> 'Successful sudo to ROOT executed.'
User: root
Sep 23 18:58:22 parrot sudo[100021]: root : TTY=pts/2 ; PWD=/root ; USER=root ; COMMAND=/usr/bin/tail -f /var/ossec/logs/alerts/alerts.log
tty: pts/2
pwd: /root
Command: /usr/bin/tail -f /var/ossec/logs/alerts/alerts.log

** Alert 1790182704.948541: ~ pam,syslog,authentication_success,pci_dss_10.2.5,pgp13.7.8,pgp13.7.9,gdpr_IV_32.2,hipaa_164.312.b,nist_800_53_AU.14,nist_800_53_AC.7,tsc_CC6.8,tsc_CC7.2,t
sc_CC7.3,
2026 Sep 23 18:58:24 parrot->journald
Rule: 5501 (level 3) -> 'PAM: Login session opened.'
User: root
Sep 23 18:58:22 parrot sudo[100021]: pam_unix(sudo:session): session opened for user root(uid=0) by atcante(uid=0)
uid: 0

** Alert 1790182750.948960: ~ perf_monitor,cpu_high,
2026 Sep 23 18:59:10 (DESKTOP-BL7DLJR) any->C:\Program Files (x86)\ossec-agent\perf-monitor.log
Rule: 100101 (level 5) -> 'CPU elevada: 83% - proceso dominante: powershell#1'
{"integration":"perf_monitor","cpu_pct":83,"mem_used_pct":38,"mem_free_mb":5042,"disk_pct":0,"disk_kbps":0,"net_kbps":0,"top_proc":"powershell#1","top_proc_cpu":393}
integration: perf_monitor
cpu_pct: 83
mem_used_pct: 38
mem_free_mb: 5042
disk_pct: 0
disk_kbps: 0
net_kbps: 0
top_proc: powershell#1
top_proc_cpu: 393
```

Figura 8. alerts.log en Parrot: alerta 100101 «CPU elevada: 83 % - proceso dominante: powershell#1» con todos los campos decodificados.

Tras cinco muestras seguidas del mismo proceso saltó la 100110 con nivel 12 y la técnica T1496 asociada (Figura 1). Al detener la carga, las alertas dejan de aparecer en la siguiente muestra.

6.3 Observaciones de la prueba

- **La CPU de la VM no llega al 95 %.** Aunque el script ocupaba todos los núcleos, la máquina virtual se quedó por debajo del 92 %. Por eso las muestras caían en la 100101 y no en la 100102, lo que me llevó a cambiar la regla 100110 (apartado 7.1).
- **top_proc_cpu supera el 100 %.** En las capturas aparecen valores entre 317 y 427. El contador por proceso de Windows suma el uso de todos los núcleos lógicos, así que con 6 núcleos el máximo es 600 %. No afecta a las reglas porque no usan ese campo, pero para que se entienda mejor habría que dividirlo por el número de núcleos.
- **El detalle de la alerta es útil para investigar.** El campo `previous_output` de la 100110 guarda las muestras anteriores que la dispararon, así que un analista puede ver de un vistazo cómo evolucionó la carga.

6.4 Qué demuestra y qué no

Las pruebas demuestran que la cadena completa funciona y que la correlación distingue una carga sostenida de un pico aislado, con una detección en torno a los cinco minutos. No demuestran, en cambio, que detecte cualquier minero real: muchos limitan su consumo al 50–70 % para pasar desapercibidos y quedarían por debajo del umbral actual. Tampoco tengo todavía una tasa de falsos positivos medida en uso normal, que es el siguiente paso del proyecto, y más ahora que el umbral efectivo de la correlación es el 80 %.

7. Problemas encontrados y cómo los resolví

Durante el proyecto me he encontrado con bastantes problemas; varios de ellos fallaban sin dar ningún error, lo que hizo que me costara más encontrarlos. La tabla los resume, de más reciente a más antiguo, y a continuación explico cada uno.

#	Problema	Fase	Estado
7.1	La 100110 no saltaba: la CPU de la VM no llegaba al 95 %	Pruebas de carga	Resuelto
7.2	El script se ejecutaba pero el log no se actualizaba	Telemetría	Resuelto
7.3	Riesgo de ejecutar PowerShell de 32 bits desde el agente	Telemetría	Resuelto
7.4	Horas distintas entre alerts.log, el dashboard y los logs del sistema	Pruebas de carga	Mitigado
7.5	Tildes rotas en la salida de los scripts de prueba	Pruebas de carga	Pendiente (menor)
7.6	El proceso dominante cambiaba de nombre entre muestras	Pruebas de carga	Resuelto
7.7	Reglas que no se disparaban nunca	Reglas	Resuelto
7.8	El manager recibía las muestras pero no las parseaba	Telemetría	Resuelto
7.9	El script no devolvía datos en un Windows en español	Telemetría	Resuelto
7.10	La contraseña de enrolamiento dejaba de funcionar	Instalación	Resuelto

7.1 La regla de correlación no saltaba: la CPU de la VM no llegaba al 95 %

Estado: **Resuelto**

Síntoma. Con la prueba de carga en marcha, la telemetría y el Administrador de tareas mostraban la CPU muy alta, pero solo aparecían alertas 100101. La 100110 no se disparaba nunca.

Causa. La 100110 estaba escrita para correlacionar únicamente la 100102 (95–100 %). En la máquina virtual, aunque el script ocupa todos los núcleos, la CPU se quedaba entre el 80 y el 91 %, probablemente por cómo el hipervisor reparte los núcleos virtuales. Ninguna muestra llegaba a la 100102.

Solución. Añadí el grupo `cpu_high` a las reglas 100101 y 100102 y cambié la 100110 para que use `<if_matched_group>cpu_high</if_matched_group>`. Ahora cuentan todas las muestras por encima del 80 %. Con este cambio la 100110 saltó en la siguiente prueba (Figura 1). La consecuencia es que el umbral efectivo baja al 80 %, lo que tengo en cuenta en las limitaciones.

7.2 El script se ejecutaba cada minuto, pero el log no se actualizaba

Estado: **Resuelto**

Síntoma. En el `ossec.log` del agente se veía que el wodle arrancaba el script cada 60 segundos, pero `perf-monitor.log` no cambiaba y al manager no llegaba nada. No había ningún error: el XML era válido y el wodle funcionaba.

Causa. El agente mantiene `perf-monitor.log` abierto de forma continua para leerlo (en el arranque aparece como `Analyzing file`). `Add-Content`, que era lo que usaba el script, pide un modo de acceso incompatible con ese fichero abierto y falla con el error «El proceso no puede obtener acceso al archivo porque está siendo utilizado en otro proceso». Como el script tiene `$ErrorActionPreference = "SilentlyContinue"` (para que el fallo de una clase CIM no tire todo el muestreo), ese error quedaba totalmente oculto. Además, al ejecutar el script a mano todo funcionaba, porque el servicio del agente no estaba leyendo el fichero en ese momento.

Solución. Abrir el fichero con `[System.IO.File]::Open(...)` pidiendo `FileShare.ReadWrite` (apartado 5.1). Con eso la escritura del script y la lectura del agente conviven sin problemas.

Cómo llegué hasta aquí

1. Confirmar que el wodle se ejecuta: filtrar `ossec.log` por «perf-monitor» y comprobar que aparece `Starting command` cada 60 s.
2. Comprobar si el fichero de salida cambia con `(Get-Item $logfile).LastWriteTime`.
3. Poner temporalmente `ignore_output` en `no` para ver la salida del script en `ossec.log`. En mi caso no apareció nada por el `SilentlyContinue`.
4. Guardar la variable automática `$Error` en un fichero de depuración al final del script. Ahí apareció el error de acceso al fichero.
5. Volver a poner `ignore_output` en `yes` para no duplicar eventos.

7.3 Riesgo de ejecutar PowerShell de 32 bits desde el agente

Estado: **Resuelto**

Síntoma. Revisando el problema anterior me di cuenta de que el wodle llamaba simplemente a `PowerShell.exe`, sin ruta.

Causa. El agente de Wazuh para Windows es un proceso de 32 bits (se instala en `Program Files (x86)`). Sin una ruta explícita, Windows puede redirigir la llamada, mediante WOW64, al PowerShell de 32 bits, y el comportamiento de algunos contadores podría no ser el mismo.

Solución. Indicar la ruta completa `C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe` y añadir un `timeout` de 30 segundos al wodle para que una ejecución colgada no se acumule con la siguiente (apartado 5.2).

7.4 Horas distintas entre alerts.log, el dashboard y los logs del sistema

Estado: **Mitigado**

Síntoma. Al revisar las alertas, la cabecera de `alerts.log` marcaba las 18:58 mientras que el log de sudo de esa misma acción marcaba las 16:58 (Figura 8). Si las horas de las máquinas no cuadran, las alertas pueden quedar fuera del rango de tiempo del dashboard y parecer que no han llegado.

Causa. Una diferencia exacta de dos horas apunta a que no todos los componentes usan la misma zona horaria (UTC frente a la hora peninsular). No lo he confirmado del todo.

Solución. Para las pruebas, compruebo siempre las alertas directamente en `alerts.log` y amplío el rango de tiempo del dashboard. Queda pendiente revisar la zona horaria y la sincronización de ambas máquinas con `timedatectl` en Parrot y `w32tm /query /status` en Windows.

7.5 Tildes rotas en la salida de los scripts de prueba

Estado: **Pendiente (menor)**

Síntoma. El script de carga muestra «ÑÃ°cleos» en lugar de «Núcleos» (Figura 5).

Causa. El script está guardado en UTF-8 sin BOM y PowerShell 5.1 lo interpreta con la codificación antigua de Windows.

Solución. Guardar los scripts de prueba en UTF-8 con BOM o evitar las tildes en los mensajes. Solo afecta a la consola, no a la telemetría, que se escribe en ASCII.

7.6 El proceso dominante cambiaba de nombre entre muestras

Estado: **Resuelto**

Síntoma. La primera prueba de carga ponía la CPU al 100 %, pero la correlación no acumulaba cinco coincidencias.

Causa. Esa prueba lanzaba cuatro procesos de PowerShell independientes. Windows numera las instancias con el mismo nombre (powershell, powershell#1, powershell#2), así que el proceso dominante iba cambiando de una muestra a otra y `same_field` nunca contaba cinco seguidas.

Solución. Rehacer la prueba con un único proceso multihilo, que además se parece más a un minero real.

7.7 Reglas que no se disparaban nunca

Estado: **Resuelto**

Síntoma. Las reglas de umbral se cargaban sin errores, pero no generaban ninguna alerta.

Causa. Wazuh compara texto, no números, de modo que una condición del tipo «CPU mayor que 80» no existe como tal.

Solución. Traducir cada umbral a una expresión regular anclada (apartado 4.3). Desde entonces, cada regla tiene su muestra de prueba en wazuh-logtest antes de darla por buena.

7.8 El manager recibía las muestras pero no las parseaba

Estado: **Resuelto**

Síntoma. Los eventos llegaban al manager, pero los campos no se decodificaban.

Causa. El log se escribía en UTF-8 y PowerShell 5.1 añade un BOM al principio del fichero. Esos bytes invisibles entran en el primer JSON y lo invalidan.

Solución. Escribir el log en ASCII (apartado 4.2).

7.9 El script no devolvía datos en un Windows en español

Estado: **Resuelto**

Síntoma. El script de ejemplo de la documentación oficial no devolvía ningún valor y tampoco mostraba errores.

Causa. Las rutas de los contadores de rendimiento están traducidas al idioma del sistema.

Solución. Usar las clases CIM, cuyas propiedades mantienen el nombre en inglés en cualquier idioma (apartado 4.1).

7.10 La contraseña de enrolamiento dejaba de funcionar

Estado: **Resuelto**

Síntoma. El agente no conseguía registrarse con la contraseña que había anotado.

Causa. El manager genera una contraseña aleatoria que cambia cada vez que se reinicia el servicio.

Solución. Obtenerla justo antes de instalar el agente o fijar una propia en `authd.pass`.

8. Limitaciones y siguientes pasos

El proyecto funciona, pero hay varios huecos que tengo identificados. Los ordeno por prioridad:

1. **Calibrar los umbrales.** Los valores actuales (CPU 80/95 %, memoria por debajo de 1 GB, red 10 MB/s) son un punto de partida. Ahora que la correlación empieza a contar desde el 80 %, es todavía más importante. El plan es dejar el laboratorio un par de días en uso normal, registrar qué alertas saltan sin que ocurra nada y ajustar los umbrales o el timeframe de la 100110, documentando cada falso positivo y el motivo de cada cambio.
2. **Normalizar el consumo por proceso.** `top_proc_cpu` suma todos los núcleos y puede superar el 100 %. Dividirlo por el número de núcleos lógicos haría el dato más fácil de leer y permitiría usarlo en las reglas.
3. **Identificar el proceso por algo más que el nombre.** Un minero renombrado como `svchost` se vería como `svchost`. Añadir el PID y la ruta del ejecutable a la telemetría permitiría distinguir un proceso legítimo de uno que solo imita su nombre.
4. **Correlacionar con el tráfico de red.** Un minero mantiene una conexión constante con su pool, pero el volumen es pequeño y nunca llega al umbral de red. Cruzar CPU alta sostenida con una conexión saliente persistente del mismo proceso (con `Get-NetTCPConnection` por PID, o con Sysmon y su evento 3) reduciría los falsos positivos frente a una compilación o un juego.
5. **Cubrir la minería en GPU.** Un minero de GPU apenas mueve la CPU. La clase `Win32_PerfFormattedData_GPUPerformanceCounters_GPUEngine` expone el uso por motor de GPU y encaja en el mismo script, aunque en una máquina virtual sin passthrough esos contadores pueden salir vacíos.
6. **Endurecer el enrolamiento.** Queda activar `use_password` con una contraseña fija en `authd.pass` y limitar el puerto 1515 a la interfaz interna, para que solo pueda registrar agentes quien conozca la contraseña.
7. **Revisar la zona horaria y la sincronización** de las dos máquinas (apartado 7.4).
8. **Probar con más de un endpoint.** No he probado cómo se comporta con decenas de agentes ni el volumen de datos que eso supone para el indexer.

9. Tecnologías y aprendizajes

Área	Tecnologías
SIEM	Wazuh 4.14 (manager, indexer, dashboard y agente), reglas y decoders en XML
Endpoint	Windows 11, PowerShell 5.1, clases CIM/WMI de rendimiento, .NET System.IO
Sistemas y red	Parrot OS, virtualización con segmento de red aislado, ufw
Detección	MITRE ATT&CK (T1496), correlación por frecuencia y grupos, expresiones regulares
Otros	Git y documentación técnica del laboratorio

Lo que me llevo

- **La diferencia entre un umbral y una detección.** Un umbral mira un instante; una detección mira un comportamiento a lo largo del tiempo. Toda la utilidad del proyecto está en esa distinción.
- **Una regla que no da error no es una regla que funcione.** Validar con wazuh-logtest antes de esperar a que ocurra algo real ahorra mucho tiempo de depuración.
- **Silenciar errores tiene un precio.** El `SilentlyContinue` que puse para que el script fuera robusto fue justo lo que escondió el fallo de escritura del log. Ahora, cuando algo no funciona y no da error, lo primero que reviso es qué lo está silenciando.
- **El entorno de pruebas también cuenta.** La máquina virtual no se comporta como un equipo físico: la CPU nunca llegó al 95 % y tuve que adaptar la regla. Es algo que no esperaba y que ahora tengo en cuenta al diseñar pruebas.
- **El ruido es un problema de seguridad.** Un sistema que avisa de todo acaba ignorado, así que calibrar los umbrales es tan importante como escribir las reglas.
- **Documentar las decisiones y su coste,** no solo el resultado final.

Anexo: cómo reproducir el laboratorio

Resumen de los pasos; el detalle está en el README del repositorio.

En el manager (Parrot OS)

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
sudo bash ./wazuh-install.sh -a
# credenciales del dashboard
sudo tar -O -xvf wazuh-install-files.tar wazuh-install-files/wazuh-passwords.txt
# contraseña de enrolamiento (cambia en cada reinicio del servicio)
sudo grep "Random password" /var/ossec/logs/ossec.log
```

En el endpoint (Windows 11, PowerShell como administrador)

```
msiexec /i wazuh-agent-4.14.7-1.msi /q WAZUH_MANAGER="<IP-manager>" `
  WAZUH_REGISTRATION_SERVER="<IP-manager>" WAZUH_REGISTRATION_PASSWORD="<password>"
NET START WazuhSvc
# copiar perf-monitor.ps1 a C:\Program Files (x86)\ossec-agent\
# pegar el wodle y el localfile (apartado 5.2) en ossec.conf
Restart-Service WazuhSvc
```

Reglas y comprobación

```
# en el manager: añadir local_rules.xml y reiniciar
sudo systemctl restart wazuh-manager
sudo /var/ossec/bin/wazuh-logtest
sudo tail -f /var/ossec/logs/alerts/alerts.log | grep --line-buffered -E "10010|10011"
```

Prueba de carga (en Windows 11)

```
.\stress-cpu-100102-100110.ps1 -DurationMinutes 8
Get-Content "C:\Program Files (x86)\ossec-agent\perf-monitor.log" -Tail 1 -Wait
```